

SmartCivic AI — High-Level Implementation Plan

Project Architecture & Technology Stack Specifications

1. Executive Summary & Project Purpose

SmartCivic AI is an intelligent civic issue reporting, automated verification, dispatching, and media escalation platform. It streamlines municipal governance by capturing complaints through web portals and IVR telephone hotlines, auto-classifying issues using Computer Vision & NLP, detecting duplicates, mapping ward severity heatmaps, dispatching field workers, and auto-escalating stalled issues to journalists.

Core System Modules

Module	Description & Capability
Citizen Web Portal	Report issues with text, image proof, quick-select categories, GPS pinning, and live timeline tracking.
24/7 Exotel IVR Hotline	Allows non-smartphone users to call a hotline and input DTMF choices (1-6) to register complaints.
AI Vision & NLP Engine	OpenCV matrix analysis, EasyOCR geotag extraction, HSV color duplicate check, and NLP keyword priority tagging.
Authority Dashboard	Regional severity heatmaps (Red/Yellow/Green), task assignment with Leaflet worker route mapping.
Worker Mobile Module	Task queue with live route directions, status updates, and mandatory photo proof of resolution.
Journalist Escalation Engine	APScheduler 5-minute background SLA rule redirecting unopened complaints to the media portal.

2. Technology Stack Matrix

Category	Technology / Library	Usage & Description
Frontend UI	HTML5, Vanilla CSS, Vanilla JS	Glassmorphic dark design system, responsive layouts, ES6 async fetch API.
Maps & Spatial	Leaflet.js & Leaflet.heat	Interactive map tiles, ward markers, worker route mapping, severity heatmaps.
Backend API	Python 3.9+, Flask, Flask-CORS	REST API server, request routing, static frontend file serving, CORS security.
Database & ORM	Neon PostgreSQL, SQLite, SQLAlchemy	Cloud PostgreSQL database with local SQLite fallback; SQLAlchemy ORM modeling.
Computer Vision	OpenCV (opencv-python-headless)	Image matrix sweep, brightness, Laplacian contrast variance, HSV duplicate matching.
OCR & Geotagging	EasyOCR, Pillow (PIL)	EXIF GPS metadata extraction and camera watermark coordinate OCR reading.
Math & Geometry	NumPy, Haversine Formula	Image array operations; 100m proximity calculations for duplicate detection.
Scheduler & SLA	APScheduler	Background job scheduler for automated 5-minute journalist escalation checks.
Telephony / IVR	Exotel IVR & Webhooks	Phone hotline integration, DTMF keypad input parsing, automated SMS alerts.
Email Service	Python smtplib & email.mime	Automated registration & complaint status email dispatch to citizens.

3. Step-by-Step Implementation Sequence

- **Step 1 — Complaint Intake:** Citizen submits complaint via Web Portal (Form + Image + Browser GPS) or Exotel IVR hotline (DTMF keypresses 1-6).
- **Step 2 — API Gateway Parsing:** Flask routes `/api/complaints` or `/api/exotel/webhook` receive and normalize incoming payload.
- **Step 3 — Geotag & Metadata Extraction:** System checks Pillow EXIF GPS data; if missing, runs EasyOCR on lower 40% image watermark crop.
- **Step 4 — AI Vision & NLP Triage:** Heuristic NLP categorizes issue type and priority. OpenCV executes HSV matrix comparison to detect duplicate reports within 100m radius.
- **Step 5 — Persistence & Notification:** Record stored in Neon PostgreSQL; automated email/SMS dispatch sent to user.
- **Step 6 — Ward Dashboard & Dispatch:** Authority dashboard displays ward heatmap; assigns field worker with live Leaflet route mapping.
- **Step 7 — Resolution & 5-Min SLA Escalation:** Worker resolves issue with photo proof. If complaint remains unassigned > 5 mins, APScheduler automatically escalates to Journalist Portal.